

Report for Robot Vision & Navigation

Coursework 01: Graph-based Optimisation & SLAM

Swastik Sharma, Saranya Roy, Anushka Sutreja

February 2026

1 Question 1(a): Factor graph structure for GPS localisation

The graph is a chain of pose vertices connected by prediction edges, with one GPS observation edge attached to each pose vertex, where:

Pose vertices will be using the state definition in Appendix A.1, Eq. (1).

Prediction edges will be using the process model in Appendix A.2, Eq. (3) with the rotation matrix in Eq. (4).

GPS observation edges use the GPS model in Appendix A.3.

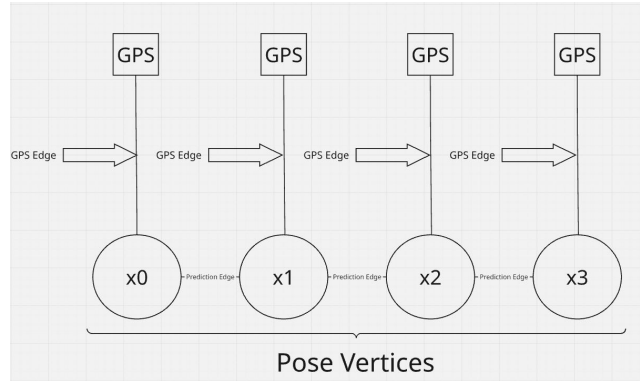


Figure 1: Pose vertices connected by prediction edges and GPS observation edges.

2 Question 1(b): PlatformPredictionEdge implementation

PlatformPredictionEdge will encode the motion constraint between subsequent pose vertices, using the process model in Appendix A.2. The edge connects $\mathbf{x}_k$ and $\mathbf{x}_{k+1}$ and uses odometry input as measurement.

2.1 Initial estimate

The `initialEstimate` method provides a starting point for optimisation by predicting the next pose from the current pose and the measured input (following the process model in Appendix A.2). We are using a small guard to avoid numerical issues when ΔT is near zero.

Code snippet

```
function initialEstimate(obj)

    xk = obj.edgeVertices{1}.x;
    theta = xk(3);

    dT = obj.dT;
    if dT < 1e-12
        obj.edgeVertices{2}.x = xk;
        return;
    end

    u = obj.z;

    M = dT * [ cos(theta) -sin(theta) 0;
               sin(theta)  cos(theta) 0;
               0           0         1];

    obj.edgeVertices{2}.x = xk + M * u;

end
```

2.2 Error function

The `computeError` method will be computing the residual used by the optimiser. The residual is defined as the motion implied by the two pose estimates(mapped into the input space) minus the measured input, making the residual close to zero when the relative motion between poses is consistent with the odometry.

$$\mathbf{e} = \mathbf{M}^{-1}(\mathbf{x}_{k+1} - \mathbf{x}_k) - \mathbf{u}.$$

Code snippet

```
function computeError(obj)

    xk = obj.edgeVertices{1}.x;
    xkp1 = obj.edgeVertices{2}.x;

    theta = xk(3);
    dT = obj.dT;

    Minv = (1/dT) * [
        cos(theta)  sin(theta)  0;
        -sin(theta) cos(theta)  0;
        0           0           1];

    u = obj.z;

    obj.errorZ = Minv * (xkp1 - xk) - u;

end
```

2.3 Linearisation

The `linearize0plus` method will provide Jacobians of the residual for the two connected pose vertices, required by Gauss-Newton to build the linear system. With the residual definition above, the Jacobians are:

$$\frac{\partial \mathbf{e}}{\partial \mathbf{x}_k} = -\mathbf{M}^{-1}, \quad \frac{\partial \mathbf{e}}{\partial \mathbf{x}_{k+1}} = \mathbf{M}^{-1}.$$

Code snippet

```
function linearize0plus(obj)

    xk = obj.edgeVertices{1}.x;

    theta = xk(3);
    dT = obj.dT;

    Minv = (1/dT) * [
        cos(theta)  sin(theta)  0;
        -sin(theta) cos(theta)  0;
        0           0           1];

    obj.J{1} = -Minv;
    obj.J{2} =  Minv;

end
```

2.4 Results

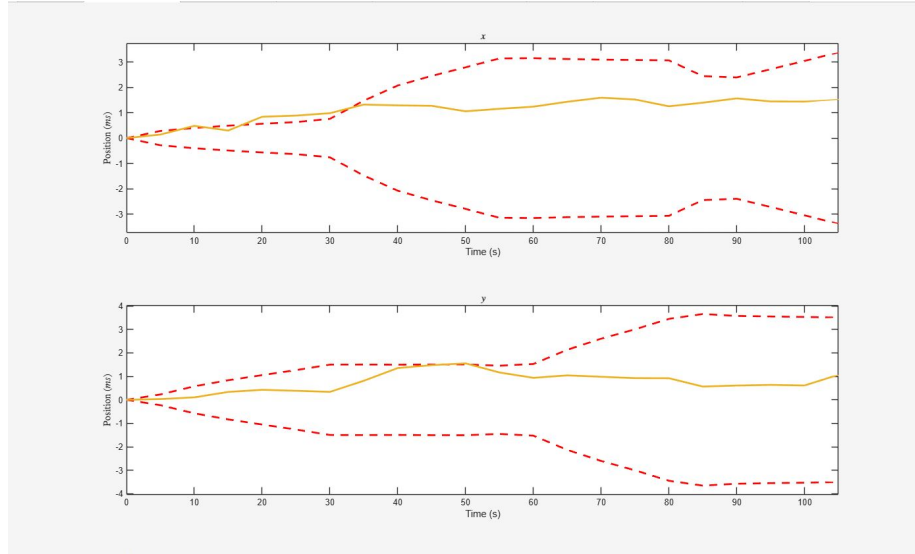


Figure 2: Position error and covariance bounds for the prediction model.

Figure 2 shows the position error in x and y together with covariance bounds. The error remains mostly within the bounds, and the bounds increase over time due to accumulated process noise, indicating that the prediction model and uncertainty propagation are consistent.

3 Question 1(c): Chi-squared and optimisation time

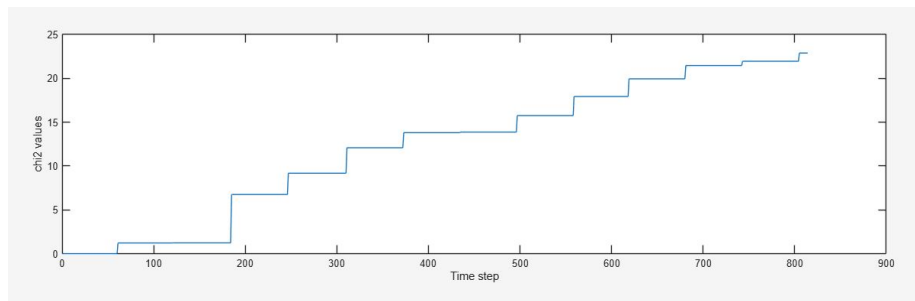


Figure 3: Chi-squared value over time.

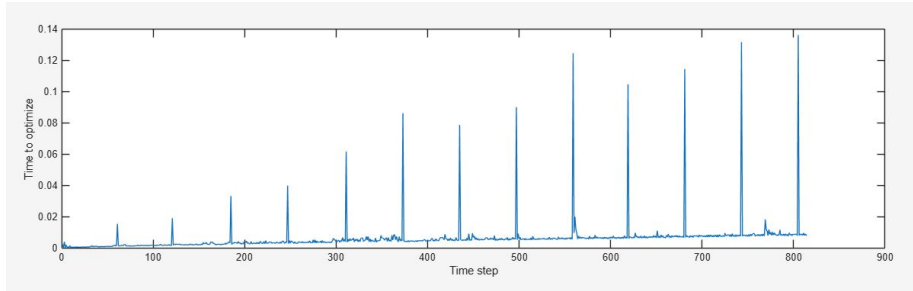


Figure 4: Optimisation time over time.

The chi-squared values increase in steps as new vertices and edges are added (Figure 3). The overall optimisation time increases and shows spikes (Figure 4) as the graph grows and each solve becomes more expensive.

4 Question 2(a): Factor graph structure for full SLAM

Full SLAM estimates both the vehicle trajectory and landmark positions. The factor graph structure is as follows:

Pose and landmark states are defined in Appendix A.1, Eq. (1) and Eq. (2).

Prediction edges use Appendix A.2.

Landmark observation edges use the range bearing model in Appendix A.4.

The graph contains a chain of pose vertices. Each landmark is represented by a landmark vertex. Whenever a landmark is observed, an observation edge will be added between the current pose and that landmark. Reobservations will create additional edges, which provide the constraints needed to optimise poses and landmarks together.

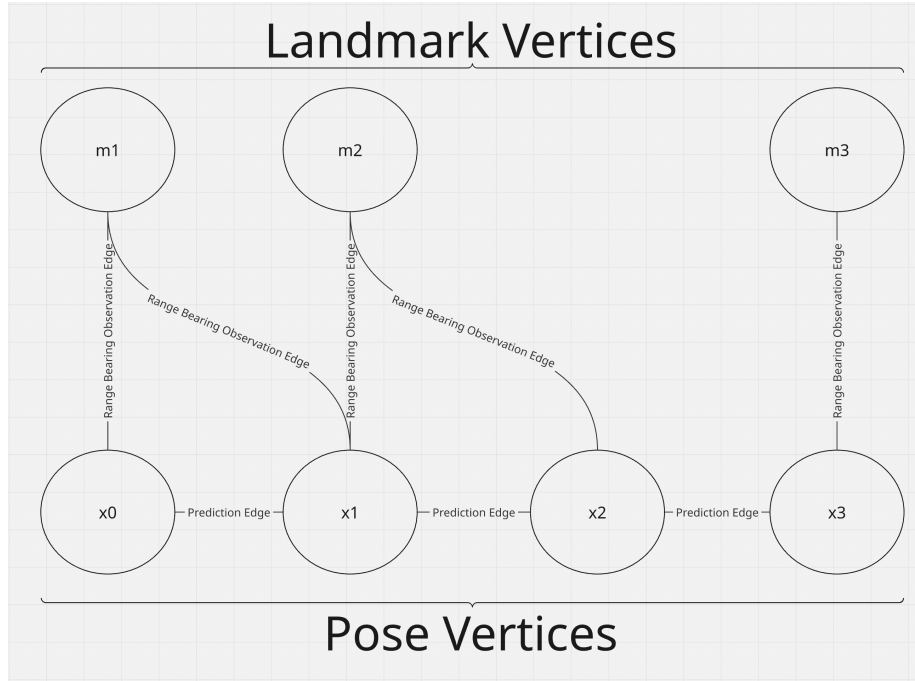


Figure 5: Full SLAM factor graph structure with pose and landmark vertices.

5 Question 2(b): LandmarkRangeBearingEdge implementation

LandmarkRangeBearingEdge connects a pose vertex and a landmark vertex, modelling a range bearing observation (Appendix A.4). The measurement is $\mathbf{z} = [r, \beta]^T$.

5.1 Initial estimate

Whenever a landmark is first observed, `initialEstimate` will initialise the landmark position in the world frame using the current pose estimate and measurement, improving convergence by avoiding poor initial landmark values.

Code snippet

```
function initialEstimate(obj)

pose = obj.edgeVertices{1}.x;
landmark = obj.edgeVertices{2}.x;

if any(isnan(landmark))
```

```

    r = obj.z(1);
    beta = obj.z(2);

    x = pose(1);
    y = pose(2);
    psi = pose(3);

    alpha = psi + beta;

    landmark = [
        x + r*cos(alpha);
        y + r*sin(alpha)];

    obj.edgeVertices{2}.x = landmark;
end

end

```

5.2 Error function

The `computeError` method will compute the predicted range and bearing from the current pose and landmark estimates (Appendix A.4), and will form the residual as measurement minus prediction. The bearing residual is wrapped to $[-\pi, \pi]$ to avoid discontinuities at $\pm\pi$.

Code snippet

```

function computeError(obj)

pose = obj.edgeVertices{1}.x;
landmark = obj.edgeVertices{2}.x;

x = pose(1);
y = pose(2);
psi = pose(3);

xi = landmark(1);
yi = landmark(2);

dx = xi - x;
dy = yi - y;

range_pred = sqrt(dx^2 + dy^2);
bearing_pred = atan2(dy, dx) - psi;

error_range = obj.z(1) - range_pred;
error_bearing = obj.z(2) - bearing_pred;

```

```

error_bearing = mod(error_bearing + pi, 2*pi) - pi;

obj.errorZ = [error_range; error_bearing];

end

```

5.3 Linearisation

The `linearize0plus` method computes Jacobians of the residual with respect to the pose and the landmark. The Jacobians will be used to linearise the measurement model during optimisation.

Code snippet

```

function linearize0plus(obj)

pose = obj.edgeVertices{1}.x;
landmark = obj.edgeVertices{2}.x;

x = pose(1);
y = pose(2);

xi = landmark(1);
yi = landmark(2);

dx = xi - x;
dy = yi - y;

q = dx^2 + dy^2;
range = sqrt(q);

J_pose = [
    dx/range,  dy/range,  0;
    -dy/q,    dx/q,      1];

J_landmark = [
    -dx/range, -dy/range;
    dy/q,     -dx/q];

obj.J{1} = J_pose;
obj.J{2} = J_landmark;

end

```

5.4 Validation and performance

In noise free configuration, the factor graph solution matches the EKF-SLAM solution, which supports the correctness of the residual and Jacobians.

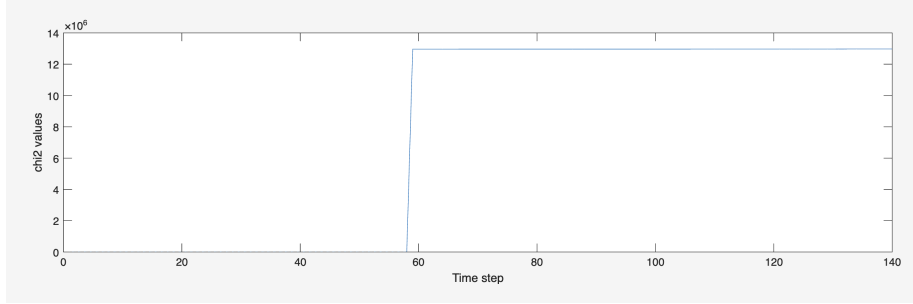


Figure 6: Chi-squared values for the SLAM system.

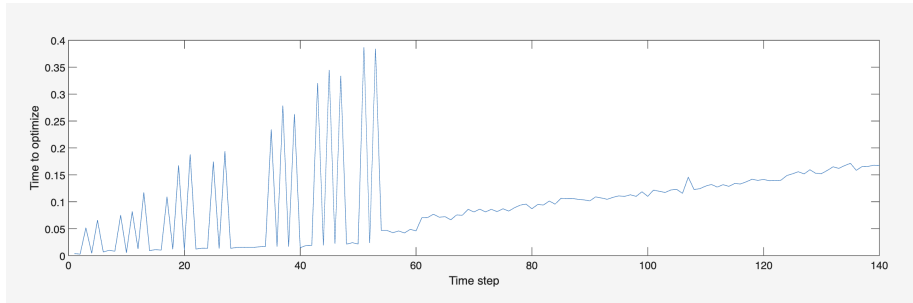


Figure 7: Optimisation time for the SLAM system.

Figures 6 and 7 show that chi-squared and optimisation time increase as the graph grows, and additional landmark constraints are introduced.

6 Question 2(c)

6.1 Results

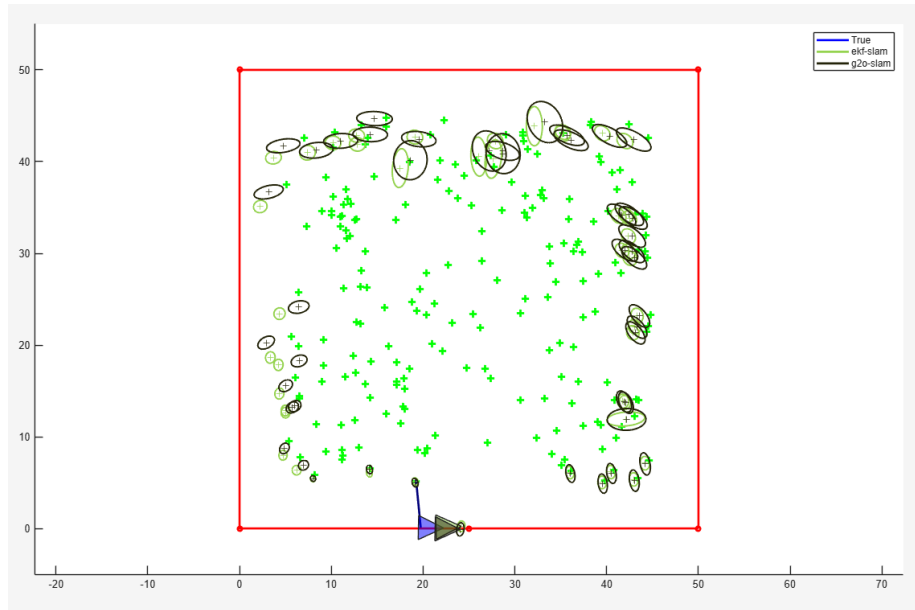


Figure 8: Map and landmark estimates for EKF SLAM and graph SLAM

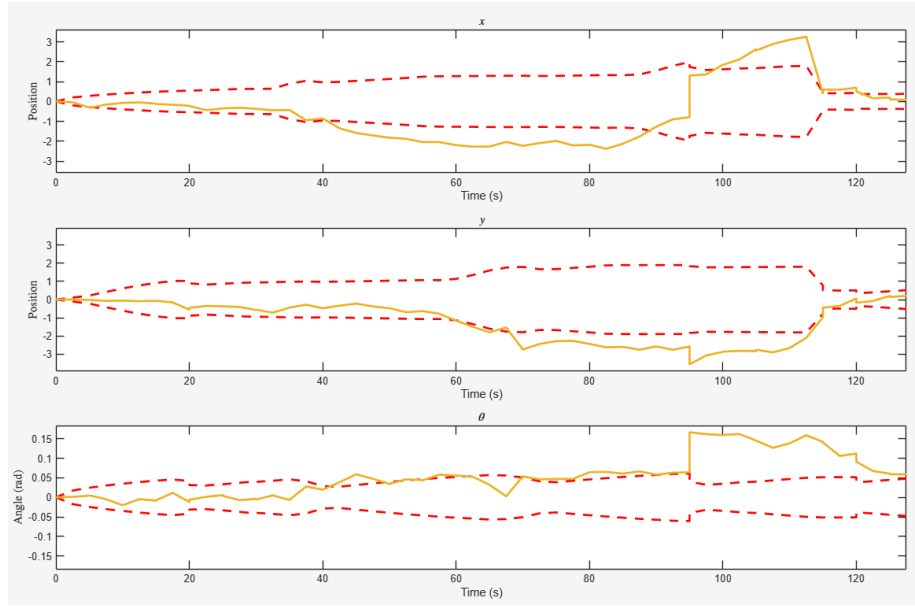


Figure 9: State estimation errors and covariance bounds for EKF SLAM

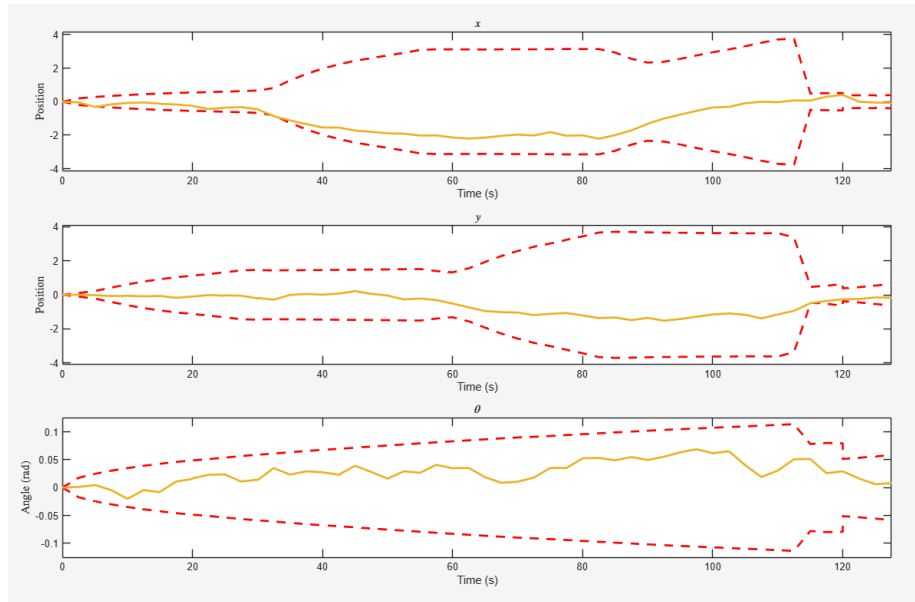


Figure 10: State estimation errors and covariance bounds for graph SLAM

6.2 Consistency comparison

The EKF SLAM solution is inconsistent in this scenario, with landmark errors that are not well represented by the covariance ellipses. The factor graph SLAM solution remains more consistent because it jointly optimises over all poses and landmarks.

6.3 Effect of detection range

A smaller detection range reduces the number of landmark observations and reobservations, which reduces the number of constraints that correct drift. This affects EKF-SLAM more significantly because it propagates errors forwards and relies on local linearisation, whereas the factor graph uses global optimisation.

7 Question 3(a): Landmark Observation Pruning

The platform motion model is implemented using the platform prediction edge defined in Appendix A.3.

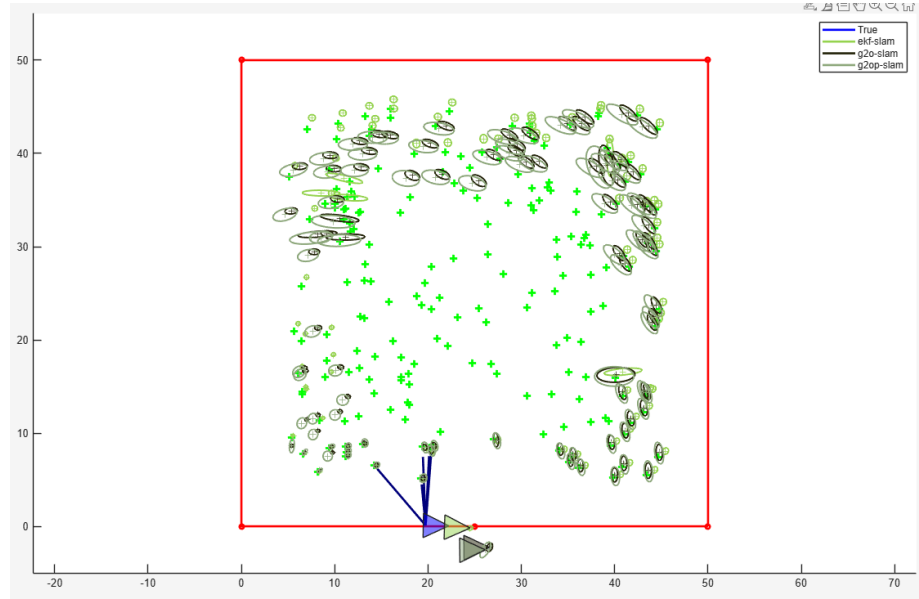


Figure 11: Map and trajectory estimates for EKF-SLAM, g2o-SLAM, and g2op-SLAM

7.1 Trajectory and map accuracy

Figure 11 shows that both graph-based methods produce accurate trajectory and landmark estimates. The estimated poses closely match the true trajectory and the landmark estimates are consistent.

The EKF-SLAM solution shows larger errors and less accurate landmark estimates. Some landmark estimates are displaced and the uncertainty is less consistent.

The pruned graph produces results very similar to the original graph. This shows that pruning does not significantly reduce estimation accuracy.

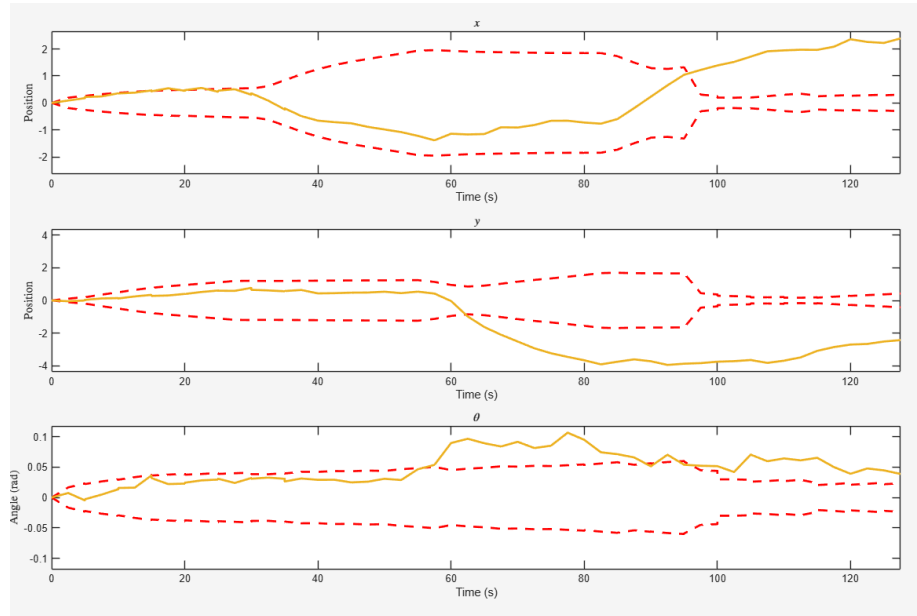


Figure 12: State estimation errors for original graph SLAM

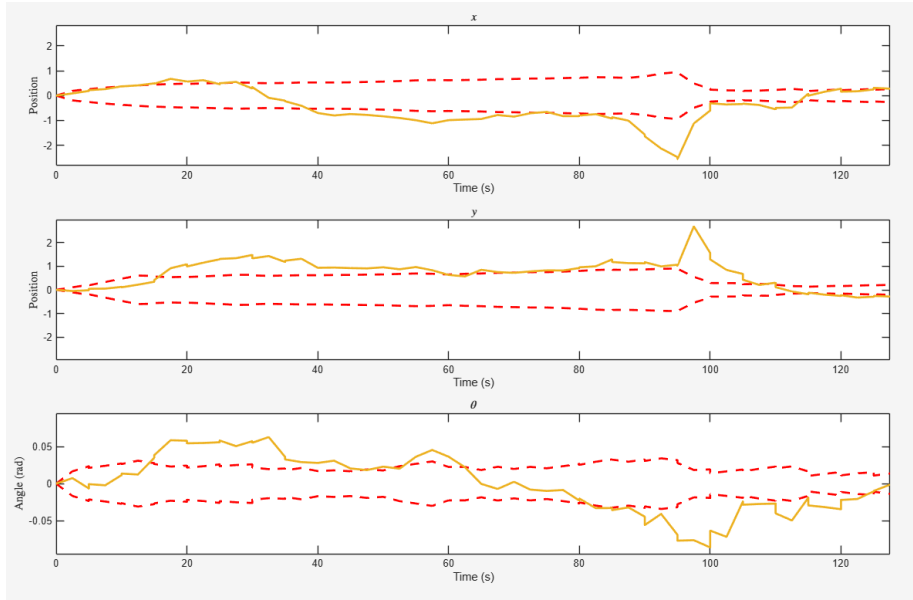


Figure 13: State estimation errors for pruned graph SLAM

7.2 State estimation errors

Figures 12 and 13 show the state errors.

Both graph-based methods maintain small errors within the covariance bounds, indicating consistent estimation.

The pruned graph has slightly larger covariance bounds. This occurs because fewer landmark observation edges are available to constrain the solution.

However, the estimation accuracy remains good.

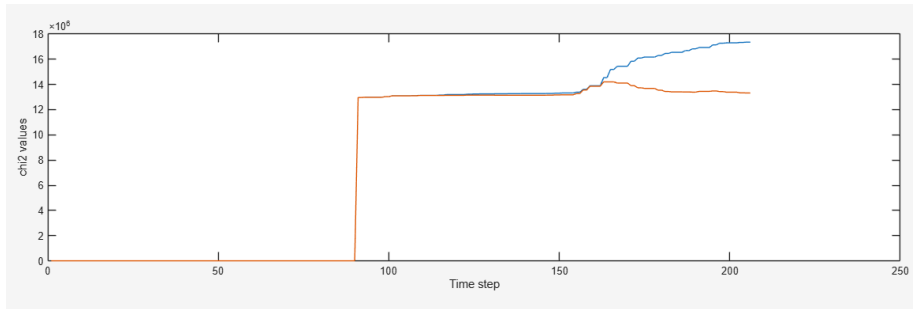


Figure 14: Chi-squared values

7.3 Chi-squared values

Figure 14 shows that both graph-based methods have very similar chi-squared values.

This indicates that pruning does not significantly affect the optimisation quality.

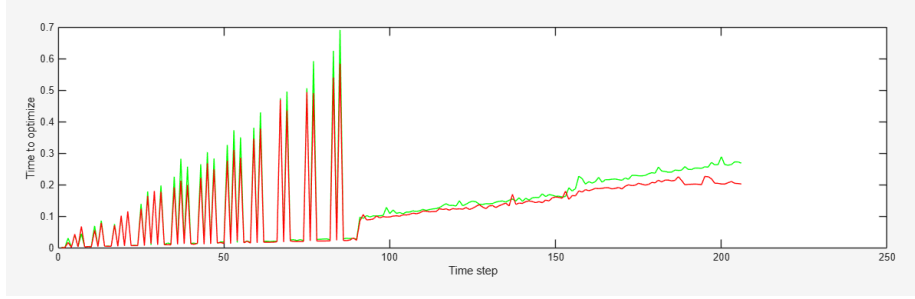


Figure 15: Optimisation time

7.4 Optimisation time

Figure 15 shows that the pruned graph requires less optimisation time.

This occurs because pruning reduces the number of observation edges, which reduces the size of the optimisation problem.

The difference becomes larger as the graph grows.

7.5 Graph structure

Both graph systems contain the same pose and landmark vertices.

The difference is that the pruned graph contains fewer observation edges.

This reduces computational cost while maintaining accuracy.

7.6 Conclusion

Pruning landmark observation edges improves computational efficiency while maintaining accurate and consistent estimates.

This approach is effective because many landmark observations are redundant.

8 Question 3(b): Comparison of g2o-SLAM and g2op-SLAM

This experiment compares EKF-SLAM, g2o-SLAM, and g2op-SLAM using the platform prediction edge defined in Appendix A.3.

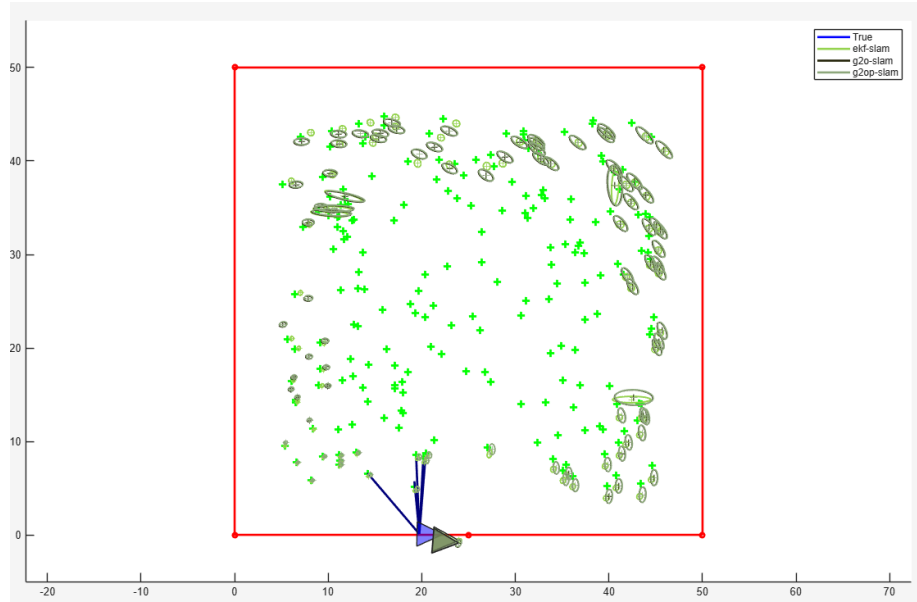


Figure 16: Map and trajectory estimates

8.1 Trajectory and map accuracy

Figure 16 shows that both graph-based methods produce accurate trajectory and landmark estimates.

The estimated trajectory closely matches the true trajectory and the landmark estimates are consistent.

EKF-SLAM produces larger errors and less accurate landmark estimates.

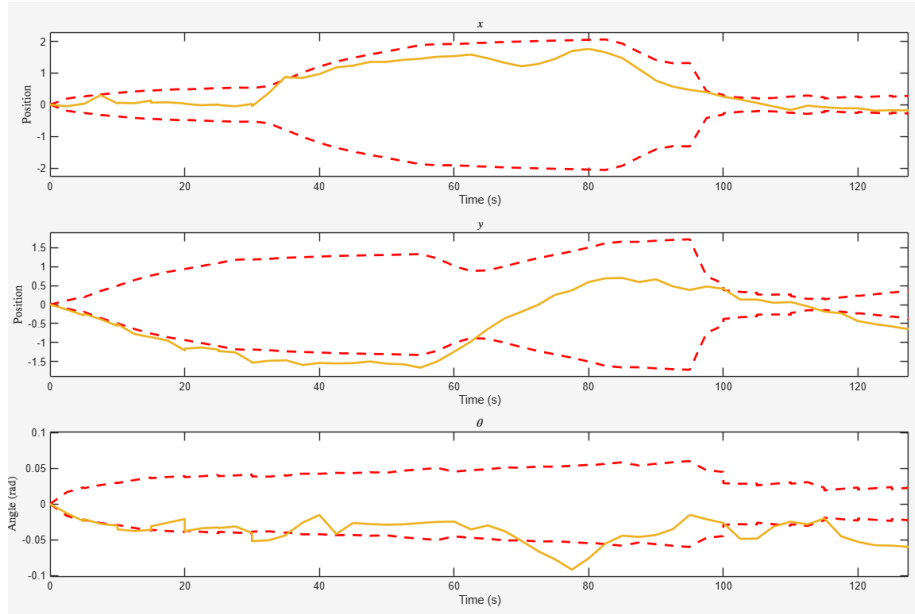


Figure 17: State estimation errors

8.2 State estimation errors

Figure 17 shows that both graph-based methods maintain small errors within the covariance bounds.

This indicates consistent estimation.

EKF-SLAM shows larger errors and less consistent uncertainty.

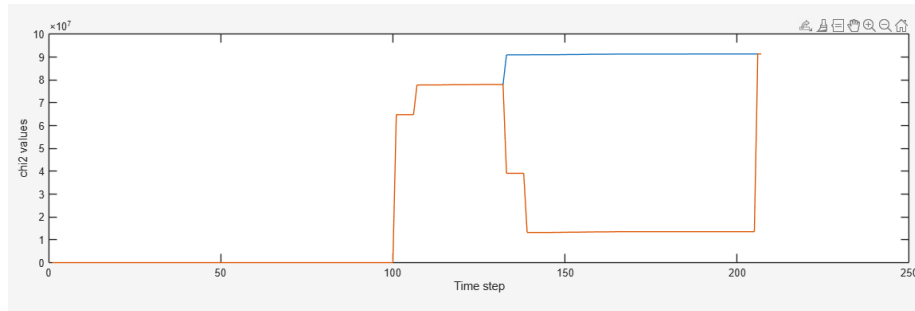


Figure 18: Chi-squared values

8.3 Chi-squared values

Figure 18 shows that both graph-based methods have nearly identical chi-squared values.

This indicates that both methods produce solutions of similar accuracy.

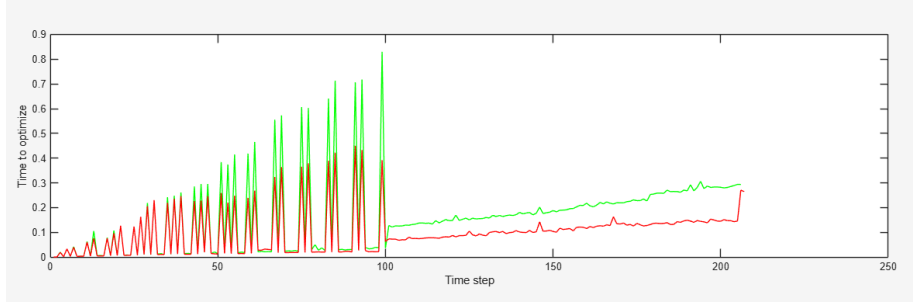


Figure 19: Optimisation time

8.4 Optimisation time

Figure 19 shows that g2op-SLAM requires less optimisation time than g2o-SLAM.

The difference becomes significant as the graph grows.

This improvement occurs because the platform prediction edge defined in Appendix A.3 provides a more efficient motion constraint.

8.5 Conclusion

Both graph-based methods produce accurate and consistent estimates.

However, g2op-SLAM is more computationally efficient.

This shows that the platform prediction edge improves optimisation efficiency without reducing estimation accuracy.